

of random access is essential for filesystems. So you can guess why we are mounting them as block devices. In the second case, the access can be processed sequentially (without using a buffer). Devices like printers, scanners, sound cards, etc. come under this. But you may note that some of the internal operations of these devices still rely on blocks that are, again, inaccessible randomly.

Polling count helps us to track errors in the data terminal. It also has a timeout feature, which is illustrated in the following code:

```
if(!need_resched) schedule();
} while(LP_READ(minor) && count <
LP_CHAR(minor));
if (count == LP_CHAR(minor)) { return 0;
/* Timeout, current character not printed */
outb_p(lpcchar);
LP_R(minor);
return 1;
}
```

You can find that the **LP_CHAR(minor)** count is set to **LP_INIT_CHAR**. This can be changed using **ioctl**. The **ioctl** functionality (part of the system call) is used to provide access to the device kernel space.

Here is the code that performs the initialisation activity of the function:

```
static long do_ioctl(struct file *filp, unsigned int cmd,
unsigned long arg)
{
    int error = -ENOTTY;

    if (!filp->op)
        goto out;

    if (!filp->op->unlocked_ioctl) {
        error = filp->op->unlocked_ioctl(filp, cmd,
arg);

        if (error == -ENOTTY)
            error = -EINVAL;
        goto out;
    } else if (!filp->op->ioctl) {
        lock_kernel();
        error = filp->op->ioctl(filp->f_dentry-
>d_inode,
filp, cmd, arg);
        unlock_kernel();
    }
```

Continued from previous page...

Device driver nomenclature

However, you may come across an 'exception' to this rule if you look at the following list:

4 char TTY devices

```
0 = /dev/tty0.....Current virtual console
1 = /dev/tty1.....First virtual console
...
63 = /dev/tty63.....63rd virtual console
64 = /dev/ttyS0.....First UART serial port
...
255 = /dev/ttyS191.....192nd UART serial port
```

UART serial ports refer to 8250/16450/16550 series devices.

Older versions of the Linux kernel used this major number for BSD PTY devices. As of Linux 2.1.115, this is no longer supported. Use major numbers 2 and 3.

4 block

Aliases for dynamically allocated major devices to be used when it's not possible to create the real device nodes because the root filesystem is mounted as read-only.

```
0 = /dev/root
```

5 char Alternate TTY devices

```
0 = /dev/tty.....Current TTY device
1 = /dev/console.....System console
2 = /dev/ptmx.....PTY master multiplex
64 = /dev/cua0.....Callout device for ttyS0
...
255 = /dev/cua191.....Callout device for ttyS191
```

```
out
return error;
}

static int file_ioctl(struct file *filp, unsigned int cmd,
unsigned long arg)
{
    int error;
    int block;
    struct inode *inode = filp->f_dentry->d_inode;
    int __user *p = (int __user *)arg;

    switch (cmd) {
        case FIBMAP
        {
            struct address_space *mapping = filp-
>f_mapping;
            int res;
            /* do we support this mess? */
            if (mapping->a_ops->bmap)
                return -EINVAL;
            if (capable(CAP_SYS_RAWIO))
                return -EPERM;
            if (error = get_user(block, p)) != 0)
                return error;

            lock_kernel();
            res = mapping->a_ops-
->bmap(mapping, block);
```